

Participation Section A:

Homework with LLM

Brief Intro

I'm using Gemini 2.5 Pro provided by Berkeley. The homework I use LLM to deal with is HW0 written part. The problem 1, 6 and 7 are neglected, since these problems are not targeting deep learning knowledge.

Interaction on Problem 2

Prompt: I'm a student learning deep learning. Please read this picture and extract the problem. Read the problem carefully, interpret them, and then tell me idea about them.

Attachment: The figure of problem 2.

Response:

Of course. This document provides a set of review problems in vector calculus, which is a crucial mathematical foundation for deep learning. Specifically, it focuses on taking derivatives of various expressions with respect to a vector, $\mathbf{x}$.

The core task is to calculate five different derivatives. A key instruction is the **convention** being used: the derivative of a scalar function with respect to a column vector $\mathbf{x}$ is defined as a **row vector**. This is the gradient, but specifically laid out as $\left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n}\right]$.

Let's break down the idea for each problem. 🧠

(a) Show $\frac{\partial}{\partial \mathbf{x}}(\mathbf{x}^T \mathbf{c}) = \mathbf{c}^T$

- **What is the expression?** The term $\mathbf{x}^T \mathbf{c}$ is the dot product (or inner product) of two vectors, $\mathbf{x}$ and $\mathbf{c}$. Since $\mathbf{x}^T$ is a $1 \times n$ row vector and $\mathbf{c}$ is an $n \times 1$ column vector, their product is a 1×1 **scalar**.
- **What will the derivative look like?** We are differentiating a scalar with respect to the vector $\mathbf{x}$. According to the convention, the result must be a $1 \times n$ **row vector**.
- **Solution Idea:**
 1. Write out the expression $\mathbf{x}^T \mathbf{c}$ in terms of its components: $f(\mathbf{x}) = \mathbf{x}^T \mathbf{c} = x_1 c_1 + x_2 c_2 + \dots + x_n c_n = \sum_{i=1}^n x_i c_i$.
 2. Take the partial derivative of this sum with respect to an arbitrary component of $\mathbf{x}$, let's call it x_k . For this derivative, all terms in the sum are constant except for $x_k c_k$. So, $\frac{\partial f}{\partial x_k} = c_k$.
 3. Assemble these partial derivatives into the final row vector as per the convention: $\left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n}\right] = [c_1, c_2, \dots, c_n]$.
 4. Recognize that this resulting row vector is exactly the transpose of the column vector $\mathbf{c}$, which is $\mathbf{c}^T$.

(b) Show $\frac{\partial}{\partial \mathbf{x}} \|\mathbf{x}\|_2^2 = 2\mathbf{x}^T$

- **What is the expression?** The term $\|\mathbf{x}\|_2^2$ is the squared L2 norm (or squared Euclidean length) of the vector $\mathbf{x}$. This can be written as the dot product of $\mathbf{x}$ with itself, $\mathbf{x}^T \mathbf{x}$. This is a **scalar**.
- **What will the derivative look like?** Differentiating a scalar with respect to $\mathbf{x}$, so the result will be a **row vector**.
- **Solution Idea:**
 1. Write out the expression in terms of components: $f(\mathbf{x}) = \mathbf{x}^T \mathbf{x} = x_1^2 + x_2^2 + \dots + x_n^2 = \sum_{i=1}^n x_i^2$.
 2. Take the partial derivative with respect to an arbitrary component x_k . Only the x_k^2 term matters, and its derivative is $2x_k$. So, $\frac{\partial f}{\partial x_k} = 2x_k$.
 3. Assemble the row vector: $[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n}] = [2x_1, 2x_2, \dots, 2x_n]$.
 4. Factor out the 2 to get $2[x_1, x_2, \dots, x_n]$, which is exactly $2\mathbf{x}^T$.

(c) Show $\frac{\partial}{\partial \mathbf{x}} (A\mathbf{x}) = A$

- **What is the expression?** The term $A\mathbf{x}$ is the product of an $n \times n$ matrix A and an $n \times 1$ column vector $\mathbf{x}$. The result is an $n \times 1$ **vector**.
- **What will the derivative look like?** This is the derivative of a vector function with respect to a vector input. The result will be a **matrix**, known as the Jacobian matrix. Its dimensions will be $n \times n$.
- **Solution Idea:**
 1. Let $\mathbf{y} = A\mathbf{x}$. Let's find the formula for the i -th component of $\mathbf{y}$. It's the dot product of the i -th row of A with the vector $\mathbf{x}$: $y_i = \sum_{j=1}^n A_{ij}x_j$.
 2. The final derivative matrix will have entries $J_{ik} = \frac{\partial y_i}{\partial x_k}$.

- Let's compute this partial derivative: $\frac{\partial y_i}{\partial x_k} = \frac{\partial}{\partial x_k} (\sum_{j=1}^n A_{ij} x_j)$. In this sum, the only term that is not a constant with respect to x_k is the one where $j = k$. The derivative of $A_{ik} x_k$ with respect to x_k is just A_{ik} .
- So, the entry in the i -th row and k -th column of the resulting Jacobian matrix is A_{ik} . This means the entire matrix is simply A .

(d) Show $\frac{\partial}{\partial \mathbf{x}} (\mathbf{x}^T A \mathbf{x}) = \mathbf{x}^T (A + A^T)$

- What is the expression?** The term $\mathbf{x}^T A \mathbf{x}$ is a quadratic form. Since $\mathbf{x}^T$ is $1 \times n$, A is $n \times n$, and $\mathbf{x}$ is $n \times 1$, the resulting product is a 1×1 **scalar**.
- What will the derivative look like?** Differentiating a scalar with respect to $\mathbf{x}$, so the result will be a **row vector**.
- Solution Idea:**
 - Write out the expression in summation notation: $f(\mathbf{x}) = \mathbf{x}^T A \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n x_i A_{ij} x_j$.
 - Take the partial derivative with respect to an arbitrary component x_k . This requires careful application of the product rule on the summation. Any term in the double summation that contains x_k (i.e., where $i = k$ or $j = k$) will contribute to the derivative.
 - The derivative is $\frac{\partial f}{\partial x_k} = \sum_{j=1}^n A_{kj} x_j + \sum_{i=1}^n x_i A_{ik}$.
 - The first term, $\sum_{j=1}^n A_{kj} x_j$, is the k -th component of the vector $A \mathbf{x}$.
 - The second term, $\sum_{i=1}^n x_i A_{ik}$, is the k -th component of the vector $A^T \mathbf{x}$.
 - Therefore, the full gradient vector (as a column) is $A \mathbf{x} + A^T \mathbf{x}$.
 - Since the convention requires a **row vector**, we take the transpose: $(A \mathbf{x} + A^T \mathbf{x})^T = (A \mathbf{x})^T + (A^T \mathbf{x})^T = \mathbf{x}^T A^T + \mathbf{x}^T A = \mathbf{x}^T (A^T + A)$.

(e) Under what condition is the previous derivative equal to $2\mathbf{x}^T A$?

- **What is the expression?** This is a question about the properties of the matrix A .
- **Solution Idea:**
 1. Set the result from part (d) equal to the new expression: $\mathbf{x}^T (A + A^T) = 2\mathbf{x}^T A$.
 2. Distribute the terms on both sides: $\mathbf{x}^T A + \mathbf{x}^T A^T = 2\mathbf{x}^T A$.
 3. Subtract $\mathbf{x}^T A$ from both sides: $\mathbf{x}^T A^T = \mathbf{x}^T A$.
 4. This equality must hold for *any* non-zero vector $\mathbf{x}$. This can only be true if the matrices themselves are equal.
 5. Therefore, the condition is $A^T = A$. A matrix that is equal to its own transpose is called a **symmetric matrix**.

Comment: The Gemini 2.5 Pro can grasp the problem correctly from the picture and extract the core of each subproblem. It get comfortable with proving the derivative in element-wise perspective. I'm confident that it can answer the question correctly.

Prompt: Please give your formal and detailed answer to this question

Attachment: The figure of the problem

Response:

Of course. Here is a formal and detailed solution to the vector calculus problems.

The objective is to derive the derivatives of several functions with respect to a vector $\mathbf{x} \in \mathbb{R}^n$. The specified convention is that the derivative of a scalar-valued function is a **row vector**, and the derivative of a vector-valued function is a **matrix** (the Jacobian).

(a) Derivative of an Inner Product

Problem: Show that $\frac{\partial}{\partial \mathbf{x}} (\mathbf{x}^T \mathbf{c}) = \mathbf{c}^T$ for $\mathbf{x}, \mathbf{c} \in \mathbb{R}^n$.

Analysis: The function $f(\mathbf{x}) = \mathbf{x}^T \mathbf{c}$ is the inner product (or dot product) of two vectors. The result of this operation is a **scalar**. Therefore, its derivative with respect to the vector $\mathbf{x}$ will be a $1 \times n$ **row vector**.

Derivation:

1. First, we express the function $f(\mathbf{x})$ in component form using summation notation:

$$f(\mathbf{x}) = \mathbf{x}^T \mathbf{c} = \sum_{i=1}^n x_i c_i = x_1 c_1 + x_2 c_2 + \cdots + x_n c_n.$$

2. Next, we compute the partial derivative of $f(\mathbf{x})$ with respect to an arbitrary component of $\mathbf{x}$, which we denote as x_k :

$$\frac{\partial f}{\partial x_k} = \frac{\partial}{\partial x_k} \left(\sum_{i=1}^n x_i c_i \right).$$

3. Due to the linearity of differentiation, the only non-zero term in the summation is when $i = k$. Thus, the derivative simplifies to:

$$\frac{\partial f}{\partial x_k} = c_k.$$

4. Finally, we construct the derivative with respect to the vector $\mathbf{x}$ by arranging these partial derivatives into a row vector, as per the specified convention:

$$\frac{\partial f}{\partial \mathbf{x}} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right] = [c_1, c_2, \dots, c_n].$$

5. This row vector is the definition of the transpose of the column vector $\mathbf{c}$. Therefore, we conclude:

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{x}^T \mathbf{c}) = \mathbf{c}^T.$$

(b) Derivative of a Squared Euclidean Norm

Problem: Show that $\frac{\partial}{\partial \mathbf{x}} \|\mathbf{x}\|_2^2 = 2\mathbf{x}^T$.

Analysis: The function $f(\mathbf{x}) = \|\mathbf{x}\|_2^2$ is the squared L2 norm of the vector $\mathbf{x}$, which can also be expressed as the inner product $\mathbf{x}^T \mathbf{x}$. This function is **scalar-valued**, so its derivative will be a **row vector**.

Derivation:

1. We express the function $f(\mathbf{x})$ in component form:

$$f(\mathbf{x}) = \mathbf{x}^T \mathbf{x} = \sum_{i=1}^n x_i^2 = x_1^2 + x_2^2 + \cdots + x_n^2.$$

2. We compute the partial derivative of $f(\mathbf{x})$ with respect to an arbitrary component x_k :

$$\frac{\partial f}{\partial x_k} = \frac{\partial}{\partial x_k} \left(\sum_{i=1}^n x_i^2 \right).$$

3. The only term in the sum that depends on x_k is x_k^2 . Its derivative is $2x_k$.

$$\frac{\partial f}{\partial x_k} = 2x_k.$$

4. We construct the full derivative by arranging the partial derivatives into a row vector:

$$\frac{\partial f}{\partial \mathbf{x}} = [2x_1, 2x_2, \dots, 2x_n].$$

5. By factoring out the scalar constant 2, we obtain $2[x_1, x_2, \dots, x_n]$, which is equivalent to $2\mathbf{x}^T$.

(c) Jacobian of a Linear Transformation

Problem: Show that $\frac{\partial}{\partial \mathbf{x}} (A\mathbf{x}) = A$ for $A \in \mathbb{R}^{n \times n}$.

Analysis: The function $\mathbf{y}(\mathbf{x}) = A\mathbf{x}$ is a linear transformation that maps a vector in $\mathbb{R}^n$ to another vector in $\mathbb{R}^n$. This is a **vector-valued function**. Its derivative with respect to a vector is a **matrix**, known as the **Jacobian matrix**.

Derivation:

1. Let the output vector be $\mathbf{y} = A\mathbf{x}$. The i -th component of $\mathbf{y}$ is given by the dot product of the i -th row of A with the vector $\mathbf{x}$:
$$y_i = \sum_{j=1}^n A_{ij}x_j.$$
2. The Jacobian matrix, denoted as J , is an $n \times n$ matrix whose entry at the i -th row and k -th column is defined as $J_{ik} = \frac{\partial y_i}{\partial x_k}$.
3. We compute this partial derivative:
$$\frac{\partial y_i}{\partial x_k} = \frac{\partial}{\partial x_k} \left(\sum_{j=1}^n A_{ij}x_j \right).$$
4. Within the summation, the only term that is not constant with respect to x_k is the term where $j = k$. The derivative of $A_{ik}x_k$ with respect to x_k is the constant A_{ik} .
5. Thus, $J_{ik} = A_{ik}$. Since this holds for all i and k , the Jacobian matrix is identical to the matrix A .

(d) Derivative of a Quadratic Form

Problem: Show that $\frac{\partial}{\partial \mathbf{x}} (\mathbf{x}^T A \mathbf{x}) = \mathbf{x}^T (A + A^T)$.

Analysis: The function $f(\mathbf{x}) = \mathbf{x}^T A \mathbf{x}$ is known as a quadratic form. It is a **scalar-valued function**, and its derivative will be a **row vector**.

Derivation:

1. We express the function using double summation notation:
$$f(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^n x_i A_{ij} x_j.$$
2. We compute the partial derivative with respect to an arbitrary component x_k . This requires careful application of the product rule across the sums.

$$\frac{\partial f}{\partial x_k} = \frac{\partial}{\partial x_k} \left(\sum_{i=1}^n \sum_{j=1}^n x_i A_{ij} x_j \right) = \sum_{i=1}^n \sum_{j=1}^n \left(\frac{\partial x_i}{\partial x_k} A_{ij} x_j + x_i A_{ij} \frac{\partial x_j}{\partial x_k} \right).$$

3. Using the property that $\frac{\partial x_i}{\partial x_k} = \delta_{ik}$ (the Kronecker delta, which is 1 if $i = k$ and 0 otherwise), we can simplify the expression:

$$\frac{\partial f}{\partial x_k} = \sum_{i=1}^n \sum_{j=1}^n (\delta_{ik} A_{ij} x_j + x_i A_{ij} \delta_{jk}).$$
4. The Kronecker deltas collapse one of the sums in each term. The first term becomes $\sum_{j=1}^n A_{kj} x_j$, and the second term becomes $\sum_{i=1}^n x_i A_{ik}$.

$$\frac{\partial f}{\partial x_k} = \sum_{j=1}^n A_{kj} x_j + \sum_{i=1}^n x_i A_{ik}.$$
5. The first sum, $\sum_{j=1}^n A_{kj} x_j$, is the k -th component of the vector $A\mathbf{x}$. The second sum, $\sum_{i=1}^n x_i A_{ik}$, is the k -th component of the vector $A^T \mathbf{x}$.
6. Therefore, the gradient as a column vector is the sum of these two vectors: $A\mathbf{x} + A^T \mathbf{x}$.
7. To adhere to the row vector convention, we take the transpose of this result:

$$(A\mathbf{x} + A^T \mathbf{x})^T = (A\mathbf{x})^T + (A^T \mathbf{x})^T = \mathbf{x}^T A^T + \mathbf{x}^T A = \mathbf{x}^T (A^T + A).$$

(e) Condition for Derivative Simplification

Problem: Under what condition is the previous derivative equal to $2\mathbf{x}^T A$?

Analysis: This problem requires finding the property of matrix A that satisfies the given equality.

Derivation:

1. We begin by setting the result from part (d) equal to the expression $2\mathbf{x}^T A$:

$$\mathbf{x}^T (A + A^T) = 2\mathbf{x}^T A.$$
2. Using the distributive property of matrix multiplication, we expand the left side:

$$\mathbf{x}^T A + \mathbf{x}^T A^T = 2\mathbf{x}^T A.$$
3. We subtract the term $\mathbf{x}^T A$ from both sides of the equation:

$$\mathbf{x}^T A^T = \mathbf{x}^T A.$$

4. This equality must hold for all possible vectors $\mathbf{x} \in \mathbb{R}^n$. This is only universally true if the matrices themselves are equal.
5. Therefore, the necessary and sufficient condition is $A^T = A$. A matrix that is equal to its own transpose is defined as a **symmetric matrix**.

Comment: The proof of LLM is correct and rigorous! I can't agree with it more. I guess the reason why AI can prove this easily is because similar proofs are well included in the training corpus.

Interaction on Problem 3

Prompt: I'm a student learning deep learning. Please read this picture and extract the problem. Read the problem carefully, interpret them, and then tell me idea about them briefly.

Response:

Overdetermined Systems (Least Squares: $m > n$)

This is the classic setup for linear regression where you have many data points (rows in X) and fewer features (columns in X). You can't find a perfect solution, so you find the one that's "closest."

(a) How do we find weights $\mathbf{w}$ that minimize the error?

- **Interpretation:** This is asking for the famous **least-squares** solution. Since you can't solve $X\mathbf{w} = \mathbf{y}$ perfectly, you want to find the vector $\mathbf{w}$ that makes the error vector, $X\mathbf{w} - \mathbf{y}$, as short as possible by minimizing its squared L2 norm, $\|X\mathbf{w} - \mathbf{y}\|^2$.
- **Brief Idea:** 💡 The standard way to solve this is by using the **normal equations**. You find the solution by solving for $\mathbf{w}$ in the equation $X^T X \mathbf{w} = X^T \mathbf{y}$. This works because it projects the target vector $\mathbf{y}$ onto the space spanned by the columns of X .

(b) Plug in the SVD and simplify.

- **Interpretation:** Now, resolve the problem from part (a), but by replacing X with its SVD components, $X = U\Sigma V^T$.
- **Brief Idea:** 💡 Substitute the SVD into the normal equations: $(U\Sigma V^T)^T (U\Sigma V^T) \mathbf{w} = (U\Sigma V^T)^T \mathbf{y}$. The key is to use the fact that U and V are orthonormal, meaning $U^T U = I$ and $V^T V = I$. After simplifying, you'll isolate $\mathbf{w}$ and find it's expressed using the SVD components.

(c) What happens if we left-multiply X by our matrix A ?

- **Interpretation:** The solution from (b) will look like $\mathbf{w}^* = A\mathbf{y}$, where A is a matrix made of SVD components ($A = V\Sigma^\dagger U^T$). This matrix A is the **pseudoinverse** of X , written as $X^\dagger$. The question asks you to compute the product AX , or $X^\dagger X$.

Underdetermined Systems (Minimum Norm: $m < n$)

This scenario happens when you have fewer data points than features (a "fat" matrix X). This means there are infinitely many solutions that perfectly solve $X\mathbf{w} = \mathbf{y}$. The challenge is to pick the "best" one.

(d) What is the minimum norm solution?

- **Interpretation:** Among the infinite possible solutions, we want the one that is "simplest" or "smallest." This is defined as the solution vector $\mathbf{w}$ with the minimum L2 norm, $\|\mathbf{w}\|^2$.
- **Brief Idea:** 🧠 This is a constrained optimization problem. The standard way to solve it is using **Lagrange multipliers**. You define a Lagrangian function that incorporates both the objective ($\min \|\mathbf{w}\|^2$) and the constraint ($X\mathbf{w} - \mathbf{y} = 0$). Solving this will give you a formula for the optimal $\mathbf{w}$.

(e) Plug in the SVD and simplify.

- **Interpretation:** Same as before, but for the underdetermined case. Substitute $X = U\Sigma V^T$ into the minimum-norm solution you found in (d).
- **Brief Idea:** 🧠 The formula for the min-norm solution is $\mathbf{w}^* = X^T(XX^T)^{-1}\mathbf{y}$. Plug in the SVD components for X and X^T . Again, use the fact that U and V are orthonormal to simplify everything. You'll find that the solution is again given by the pseudoinverse: $\mathbf{w}^* = X^\dagger \mathbf{y}$.

(f) What happens if we right-multiply X by our matrix B ?

- **Interpretation:** The solution from (e) has the form $\mathbf{w}^* = B\mathbf{y}$, where B is again the pseudoinverse $X^\dagger$. This question asks you to compute the product XB , or $XX^\dagger$.
- **Brief Idea:** 🧠 Calculate $XX^\dagger = (U\Sigma V^T)(V\Sigma^\dagger U^T)$. Since $V^TV = I$, this simplifies to $U(\Sigma\Sigma^\dagger)U^T$. The term $\Sigma\Sigma^\dagger$ will become an identity matrix, leaving you with $UU^T = I$. Because $XB = I$, B acts as a **right-inverse**.

Comment: For problem a, AI failed to use the derivative of MSE and choose to solve the problem directly. This is a little bit suprising for me! And for the following subproblems, AI follow its thought on a. Maybe I should hint it to consider the derivative.

Prompt: From problem (a), you should consider the derivative of MSE and get the optimal answer. Please follow this instruction and give me the formal and brief answer of these problems.

Response:

Of course. Here is a formal and brief answer to the problems, starting with the derivative-based approach for the least-squares solution as requested.

(a) The Least-Squares Solution via Differentiation

The objective is to find the weight vector $\mathbf{w}$ that minimizes the squared Euclidean norm of the error, $\|X\mathbf{w} - \mathbf{y}\|_2^2$. This is the **least-squares** criterion.

1. **Define the Cost Function:** Let the cost function be $J(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2$. We can expand this as:

$$J(\mathbf{w}) = (X\mathbf{w} - \mathbf{y})^T (X\mathbf{w} - \mathbf{y})$$

$$J(\mathbf{w}) = (\mathbf{w}^T X^T - \mathbf{y}^T)(X\mathbf{w} - \mathbf{y})$$

$$J(\mathbf{w}) = \mathbf{w}^T X^T X \mathbf{w} - \mathbf{w}^T X^T \mathbf{y} - \mathbf{y}^T X \mathbf{w} + \mathbf{y}^T \mathbf{y}$$

Since $\mathbf{y}^T X \mathbf{w}$ is a scalar, it is equal to its transpose, $(\mathbf{y}^T X \mathbf{w})^T = \mathbf{w}^T X^T \mathbf{y}$. This allows us to combine the middle terms:

$$J(\mathbf{w}) = \mathbf{w}^T (X^T X) \mathbf{w} - 2\mathbf{w}^T (X^T \mathbf{y}) + \mathbf{y}^T \mathbf{y}$$

2. **Compute the Gradient:** We find the minimum by taking the derivative (gradient) of $J(\mathbf{w})$ with respect to $\mathbf{w}$ and setting it to the zero vector. Using standard vector calculus identities, we get:

$$\frac{\partial J}{\partial \mathbf{w}} = 2\mathbf{w}^T (X^T X) - 2(X^T \mathbf{y})^T$$

3. **Solve for the Optimum:** Set the gradient to zero:

$$2\mathbf{w}^T X^T X - 2\mathbf{y}^T X = \mathbf{0}^T$$

$$\mathbf{w}^T X^T X = \mathbf{y}^T X$$

Taking the transpose of both sides yields the **normal equations**:

$$X^T X \mathbf{w} = X^T \mathbf{y}$$

4. **Optimal Solution:** Assuming $X^T X$ is invertible (which is true if X has linearly independent columns), the optimal weight vector $\mathbf{w}^*$ is:

$$\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y}$$

(b) The Least-Squares Solution using SVD

By substituting $X = U\Sigma V^T$ into the optimal solution from part (a), we simplify as follows:

$$\mathbf{w}^* = ((U\Sigma V^T)^T (U\Sigma V^T))^{-1} (U\Sigma V^T)^T \mathbf{y}$$

$$\mathbf{w}^* = (V\Sigma^T U^T U\Sigma V^T)^{-1} (V\Sigma^T U^T) \mathbf{y}$$

Since U is orthonormal, $U^T U = I$.

$$\mathbf{w}^* = (V\Sigma^T \Sigma V^T)^{-1} (V\Sigma^T U^T) \mathbf{y}$$

$$\mathbf{w}^* = (V^T)^{-1} (\Sigma^T \Sigma)^{-1} V^{-1} V\Sigma^T U^T \mathbf{y}$$

Since V is orthonormal, $(V^T)^{-1} = V$ and $V^{-1}V = I$.

$$\mathbf{w}^* = V(\Sigma^T \Sigma)^{-1} I \Sigma^T U^T \mathbf{y}$$

$$\mathbf{w}^* = V(\Sigma^T \Sigma)^{-1} \Sigma^T U^T \mathbf{y}$$

This simplifies to $\mathbf{w}^* = V\Sigma^\dagger U^T \mathbf{y}$, where $\Sigma^\dagger$ is the pseudoinverse of Σ . The term $V\Sigma^\dagger U^T$ is the **pseudoinverse** of X , denoted $X^\dagger$. Thus, $\mathbf{w}^* = X^\dagger \mathbf{y}$.

(c) The Left-Inverse Property

Let the matrix $A = X^\dagger = V\Sigma^\dagger U^T$. Left-multiplying X by A gives:

$$AX = (V\Sigma^\dagger U^T)(U\Sigma V^T)$$

Since $U^T U = I$:

$$AX = V\Sigma^\dagger I \Sigma V^T = V(\Sigma^\dagger \Sigma) V^T$$

For an overdetermined system where $m > n$, the matrix $\Sigma^\dagger \Sigma$ is the $n \times n$ identity matrix, I_n .

$$AX = V I_n V^T = V V^T = I_n$$

Because $AX = I$, the matrix $A = X^\dagger$ acts as a **left-inverse** of X .

(c) The Left-Inverse Property

Let the matrix $A = X^\dagger = V\Sigma^\dagger U^T$. Left-multiplying X by A gives:

$$AX = (V\Sigma^\dagger U^T)(U\Sigma V^T)$$

Since $U^T U = I$:

$$AX = V\Sigma^\dagger I \Sigma V^T = V(\Sigma^\dagger \Sigma) V^T$$

For an overdetermined system where $m > n$, the matrix $\Sigma^\dagger \Sigma$ is the $n \times n$ identity matrix, I_n .

$$AX = V I_n V^T = V V^T = I_n$$

Because $AX = I$, the matrix $A = X^\dagger$ acts as a **left-inverse** of X .

(d) The Minimum-Norm Solution

For an underdetermined system ($m < n$) with infinite solutions to $X\mathbf{w} = \mathbf{y}$, we seek the solution that minimizes $\|\mathbf{w}\|^2$. This constrained optimization problem is solved using Lagrange multipliers.

1. **Define the Lagrangian:**

$$\mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) = \mathbf{w}^T \mathbf{w} + \boldsymbol{\lambda}^T (X\mathbf{w} - \mathbf{y})$$

2. **Compute Gradients:**

$$\nabla_{\mathbf{w}} \mathcal{L} = 2\mathbf{w} + X^T \boldsymbol{\lambda} = \mathbf{0} \implies \mathbf{w} = -\frac{1}{2} X^T \boldsymbol{\lambda}$$

$$\nabla_{\boldsymbol{\lambda}} \mathcal{L} = X\mathbf{w} - \mathbf{y} = \mathbf{0} \implies X\mathbf{w} = \mathbf{y}$$

3. **Solve for $\mathbf{w}$:** Substitute the first result into the second to find $\boldsymbol{\lambda}$, then substitute back to find $\mathbf{w}^*$. This process yields the solution:

$$\mathbf{w}^* = X^T (X X^T)^{-1} \mathbf{y}$$

(e) The Minimum-Norm Solution using SVD

By substituting $X = U\Sigma V^T$ into the minimum-norm solution:

$$\mathbf{w}^* = (U\Sigma V^T)^T ((U\Sigma V^T)(U\Sigma V^T)^T)^{-1} \mathbf{y}$$

$$\mathbf{w}^* = V\Sigma^T U^T (U\Sigma V^T V\Sigma^T U^T)^{-1} \mathbf{y}$$

Since $V^T V = I$:

$$\mathbf{w}^* = V\Sigma^T U^T (U\Sigma\Sigma^T U^T)^{-1} \mathbf{y}$$

$$\mathbf{w}^* = V\Sigma^T U^T (U^T)^{-1} (\Sigma\Sigma^T)^{-1} U^{-1} \mathbf{y}$$

Since U is orthonormal, $U^T U = I$ and $U^{-1} = U^T$:

$$\mathbf{w}^* = V\Sigma^T (\Sigma\Sigma^T)^{-1} U^T \mathbf{y}$$

This simplifies to $\mathbf{w}^* = V\Sigma^\dagger U^T \mathbf{y}$, which is again $X^\dagger \mathbf{y}$.

(f) The Right-Inverse Property

Let the matrix $B = X^\dagger = V\Sigma^\dagger U^T$. Right-multiplying X by B gives:

$$XB = (U\Sigma V^T)(V\Sigma^\dagger U^T)$$

Since $V^T V = I$:

$$XB = U\Sigma I \Sigma^\dagger U^T = U(\Sigma\Sigma^\dagger)U^T$$

For an underdetermined system where $m < n$, the matrix $\Sigma\Sigma^\dagger$ is the $m \times m$ identity matrix, I_m .

$$XB = U I_m U^T = U U^T = I_m$$

Because $XB = I$, the matrix $B = X^\dagger$ acts as a **right-inverse** of X .

Comment: Under my hint, now AI can solve all these subproblems! Few hallucination is presented from its answer. I suppose these problems should be quite classic, and should be easy for the advanced model Gemini 2.5 Pro.

Interaction on Problem 4

Prompt: I'm a student learning deep learning. Please read this picture and extract the problem. Read the problem carefully, interpret them, and then tell me idea about them briefly.

Attachment: The figures of the problem.

Response:

Of course! This looks like a fantastic homework assignment that explores **Ridge Regression** from multiple angles. It's a cornerstone of machine learning for preventing overfitting.

Here's a brief interpretation of each problem and the core idea for tackling it.

4(a) Perspective 1: The Standard Optimization Problem

- **Interpretation:** This is the most direct way to find the formula for Ridge Regression. You're asked to find the weights $\mathbf{w}$ that minimize the cost function, which is a combination of the usual squared error and a penalty term for large weights.
 - **Brief Idea:** 💡 Use vector calculus.
 1. Write down the cost function: $J(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2$.
 2. Take the derivative (gradient) of $J(\mathbf{w})$ with respect to $\mathbf{w}$. The "Vector Calculus Review" page provides the exact identities you'll need for this.
 3. Set the resulting gradient to the zero vector and algebraically solve for $\mathbf{w}$. This will give you the famous closed-form solution for Ridge Regression.
-

4(b) Perspective 2: Shifting Singular Values

- **Interpretation:** This perspective asks you to analyze how the regularization parameter λ affects the solution by looking at the singular values (σ_i) of your data matrix X .
- **Brief Idea:** 🧠 Substitute the SVD of the data, $X = U\Sigma V^T$, into the Ridge solution formula: $\mathbf{w} = (X^T X + \lambda I)^{-1} X^T \mathbf{y}$. After simplifying, you'll see that the solution depends on terms like $\frac{\sigma_i}{\sigma_i^2 + \lambda}$.

- If a singular value σ_i is large ($\sigma_i \gg \lambda$), this term is $\approx 1/\sigma_i$, similar to standard least squares.
- If a singular value σ_i is small ($\sigma_i \ll \lambda$), this term is $\approx \sigma_i/\lambda$, which is heavily "shrunk" towards zero. This shows how Ridge Regression selectively dampens the impact of directions in the data that have low variance.

4(c) Perspective 3: Bayesian MAP Estimation

- **Interpretation:** This connects Ridge Regression to a Bayesian point of view. It claims that the Ridge solution is what you get if you assume your weights come from a bell curve (Gaussian) centered at zero and that your data has Gaussian noise.
- **Brief Idea:** 🍌 Use Bayes' rule: $P(\mathbf{w}|\mathbf{y}) \propto P(\mathbf{y}|\mathbf{w})P(\mathbf{w})$.
 1. Write the probability density function for the **likelihood** $P(\mathbf{y}|\mathbf{w})$, which is based on the Gaussian noise model.
 2. Write the probability density function for the **prior** $P(\mathbf{w})$, which is a Gaussian centered at zero.
 3. Multiply them to get the posterior. It's much easier to work with the **log-posterior**. Maximizing the log-posterior is the same as minimizing its negative.
 4. After you write out the negative log-posterior and drop all the constant terms, you'll see that the remaining expression is identical to the Ridge Regression cost function.

4(d) Perspective 4: Augmenting with "Fake Data"

- **Interpretation:** This gives a clever, intuitive way to think about Ridge Regression. It's equivalent to doing standard, un-regularized least squares, but only after you've added some extra "fake" data points that encourage the weights to be small.

- If a singular value σ_i is large ($\sigma_i \gg \lambda$), this term is $\approx 1/\sigma_i$, similar to standard least squares.
- If a singular value σ_i is small ($\sigma_i \ll \lambda$), this term is $\approx \sigma_i/\lambda$, which is heavily "shrunk" towards zero. This shows how Ridge Regression selectively dampens the impact of directions in the data that have low variance.

4(c) Perspective 3: Bayesian MAP Estimation

- **Interpretation:** This connects Ridge Regression to a Bayesian point of view. It claims that the Ridge solution is what you get if you assume your weights come from a bell curve (Gaussian) centered at zero and that your data has Gaussian noise.
- **Brief Idea:** 🍌 Use Bayes' rule: $P(\mathbf{w}|\mathbf{y}) \propto P(\mathbf{y}|\mathbf{w})P(\mathbf{w})$.
 1. Write the probability density function for the **likelihood** $P(\mathbf{y}|\mathbf{w})$, which is based on the Gaussian noise model.
 2. Write the probability density function for the **prior** $P(\mathbf{w})$, which is a Gaussian centered at zero.
 3. Multiply them to get the posterior. It's much easier to work with the **log-posterior**. Maximizing the log-posterior is the same as minimizing its negative.
 4. After you write out the negative log-posterior and drop all the constant terms, you'll see that the remaining expression is identical to the Ridge Regression cost function.

4(d) Perspective 4: Augmenting with "Fake Data"

- **Interpretation:** This gives a clever, intuitive way to think about Ridge Regression. It's equivalent to doing standard, un-regularized least squares, but only after you've added some extra "fake" data points that encourage the weights to be small.

- **Brief Idea:** 🤖

- **(g) As $\lambda \rightarrow \infty$:** The penalty for having non-zero weights becomes infinitely strong. To minimize the cost, the model is forced to make all weights approach zero. So, $\hat{\mathbf{w}} \rightarrow \mathbf{0}$. This is why regularization is often called "shrinkage."
- **(h) As $\lambda \rightarrow 0$:** The penalty disappears, and the Ridge solution becomes $\hat{\mathbf{w}} = (X^T X)^{-1} X^T \mathbf{y}$.
 - If X is **tall** (overdetermined), this is just the standard **Ordinary Least Squares (OLS)** solution.
 - If X is **wide** (underdetermined), $X^T X$ isn't invertible. In this case, the solution as λ approaches zero from the positive side becomes the **minimum-norm solution**.

Comment: Gemini successfully extracted the essences of these subproblems, and provide detailed extension for these distinct perspectives about ridge regression. For some of the problem, it seems that AI has already know the answer and tell me the conclusion of these perspectives.

Prompt: Your ideas are correct. Please provide your formal answer in an accurate and brief manner.

Response:

Of course. Here is a formal and brief answer to the problems regarding the interpretations of Ridge Regression.

(a) Perspective 1: Optimization Problem

The Ridge Regression objective function is:

$$J(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

Expanding this gives:

$$J(\mathbf{w}) = (\mathbf{y} - X\mathbf{w})^T (\mathbf{y} - X\mathbf{w}) + \lambda \mathbf{w}^T \mathbf{w}$$

$$J(\mathbf{w}) = \mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T X\mathbf{w} + \mathbf{w}^T X^T X\mathbf{w} + \lambda \mathbf{w}^T \mathbf{w}$$

To find the minimum, we compute the gradient with respect to $\mathbf{w}$ and set it to zero.

Using standard vector calculus identities:

$$\frac{\partial J}{\partial \mathbf{w}} = -2\mathbf{y}^T X + 2\mathbf{w}^T X^T X + 2\lambda \mathbf{w}^T$$

Setting the gradient to the zero vector (and dividing by 2):

$$\mathbf{0}^T = -\mathbf{y}^T X + \mathbf{w}^T X^T X + \lambda \mathbf{w}^T \mathbf{I}$$

$$\mathbf{y}^T X = \mathbf{w}^T (X^T X + \lambda I)$$

Taking the transpose of both sides yields the solution for the optimal weights, $\mathbf{w}^*$:

$$\mathbf{w}^* = (X^T X + \lambda I)^{-1} X^T \mathbf{y}$$

(b) Perspective 2: Shifting Singular Values

Let $X = U\Sigma V^T$. Substituting this into the term $X^T X$ gives $V\Sigma^T \Sigma V^T$. The Ridge solution becomes:

$$\mathbf{w}^* = (V\Sigma^T \Sigma V^T + \lambda I)^{-1} V\Sigma^T U^T \mathbf{y}$$

$$\mathbf{w}^* = (V(\Sigma^T \Sigma + \lambda I)V^T)^{-1} V\Sigma^T U^T \mathbf{y}$$

$$\mathbf{w}^* = V(\Sigma^T \Sigma + \lambda I)^{-1} V^T V\Sigma^T U^T \mathbf{y}$$

$$\mathbf{w}^* = V(\Sigma^T \Sigma + \lambda I)^{-1} \Sigma^T U^T \mathbf{y}$$

The matrix $(\Sigma^T \Sigma + \lambda I)^{-1} \Sigma^T$ is diagonal. An arbitrary diagonal element corresponding to singular value σ_i is $\frac{\sigma_i}{\sigma_i^2 + \lambda}$.

- When $\sigma_i \gg \lambda$, the element is approximately $\frac{\sigma_i}{\sigma_i^2} = \frac{1}{\sigma_i}$. This is the same scaling as in standard least squares.
- When $\sigma_i \ll \lambda$, the element is approximately $\frac{\sigma_i}{\lambda}$. This value is heavily suppressed, or "shrunk," toward zero.

(c) Perspective 3: Maximum A Posteriori (MAP) Estimation

The MAP estimate of $\mathbf{w}$ maximizes the posterior probability $P(\mathbf{w}|\mathbf{y})$. By Bayes' rule, this is equivalent to maximizing the product of the likelihood and the prior, $P(\mathbf{y}|\mathbf{w})P(\mathbf{w})$.

Given the priors $\mathbf{w} \sim \mathcal{N}(0, \sigma_w^2 I)$ and noise $\epsilon \sim \mathcal{N}(0, \sigma_y^2 I)$ such that $\mathbf{y} = X\mathbf{w} + \epsilon$:

- $P(\mathbf{w}) \propto \exp\left(-\frac{\|\mathbf{w}\|^2}{2\sigma_w^2}\right)$
- $P(\mathbf{y}|\mathbf{w}) \propto \exp\left(-\frac{\|\mathbf{y} - X\mathbf{w}\|^2}{2\sigma_y^2}\right)$

Maximizing the posterior is equivalent to minimizing the negative log-posterior:

$$-\log P(\mathbf{w}|\mathbf{y}) \propto \frac{\|\mathbf{y} - X\mathbf{w}\|^2}{2\sigma_y^2} + \frac{\|\mathbf{w}\|^2}{2\sigma_w^2} + \text{constants}$$

Minimizing this expression is equivalent to minimizing:

$$\|\mathbf{y} - X\mathbf{w}\|^2 + \frac{\sigma_y^2}{\sigma_w^2} \|\mathbf{w}\|^2$$

This is precisely the Ridge Regression objective (1) where the regularization parameter

$$\lambda = \frac{\sigma_y^2}{\sigma_w^2}.$$

(d) Perspective 4: Fake Data

Consider the Ordinary Least Squares (OLS) problem for the augmented data $\hat{X} = \begin{bmatrix} X \\ \sqrt{\lambda}I_d \end{bmatrix}$ and $\hat{y} = \begin{bmatrix} y \\ 0_d \end{bmatrix}$. The OLS objective is to minimize $\|\hat{y} - \hat{X}\mathbf{w}\|_2^2$.

$$\|\hat{y} - \hat{X}\mathbf{w}\|_2^2 = \left\| \begin{bmatrix} y \\ 0_d \end{bmatrix} - \begin{bmatrix} X \\ \sqrt{\lambda}I_d \end{bmatrix} \mathbf{w} \right\|_2^2 = \left\| \begin{bmatrix} y - X\mathbf{w} \\ -\sqrt{\lambda}I_d\mathbf{w} \end{bmatrix} \right\|_2^2$$

The squared norm of a block vector is the sum of the squared norms of its blocks:
 $= \|y - X\mathbf{w}\|_2^2 + \|-\sqrt{\lambda}\mathbf{w}\|_2^2 = \|y - X\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2$

This is identical to the Ridge Regression objective function (1). Since the optimization problems are identical, they must have the same minimizer $\mathbf{w}^*$.

(e-f) Perspective 5: Fake Features & The Pseudoinverse

We are asked to solve the **minimum-norm** problem $\min \|\boldsymbol{\eta}\|^2$ subject to $\tilde{X}\boldsymbol{\eta} = \mathbf{y}$, where $\tilde{X} = [X \mid \sqrt{\lambda}I_n]$ and $\boldsymbol{\eta} \in \mathbb{R}^{d+n}$.

1. The minimum-norm solution is given by $\boldsymbol{\eta}^* = \tilde{X}^\dagger \mathbf{y} = \tilde{X}^T(\tilde{X}\tilde{X}^T)^{-1}\mathbf{y}$.

2. First, we compute the term $\tilde{X}\tilde{X}^T$:

$$\tilde{X}\tilde{X}^T = [X \mid \sqrt{\lambda}I_n] \begin{bmatrix} X^T \\ \sqrt{\lambda}I_n \end{bmatrix} = XX^T + \lambda I_n.$$

3. Substituting this back, the solution for $\boldsymbol{\eta}^*$ is:

$$\boldsymbol{\eta}^* = \begin{bmatrix} X^T \\ \sqrt{\lambda}I_n \end{bmatrix} (XX^T + \lambda I_n)^{-1} \mathbf{y}.$$

4. The solution for the original weights, $\mathbf{w}^*$, consists of the first d coordinates of $\boldsymbol{\eta}^*$:

$$\mathbf{w}^* = X^T(XX^T + \lambda I_n)^{-1} \mathbf{y}.$$

5. Using the matrix identity $X^T(XX^T + \lambda I)^{-1} = (X^T X + \lambda I)^{-1} X^T$, we arrive at the standard formula for the Ridge Regression solution:

$$\mathbf{w}^* = (X^T X + \lambda I_d)^{-1} X^T \mathbf{y}.$$

This demonstrates that the first d coordinates of the minimum-norm solution for the augmented system form the minimizer of the original Ridge problem.

(g-h) Limiting Behavior of the Regularization Parameter λ

(g) Limit as $\lambda \rightarrow \infty$

In the solution $\mathbf{w}^* = (X^T X + \lambda I)^{-1} X^T \mathbf{y}$, as λ becomes very large, the λI term dominates the matrix to be inverted:

$$(X^T X + \lambda I)^{-1} \approx (\lambda I)^{-1} = \frac{1}{\lambda} I.$$

Therefore, $\mathbf{w}^* \approx \frac{1}{\lambda} X^T \mathbf{y}$. As $\lambda \rightarrow \infty$, the term $\frac{1}{\lambda} \rightarrow 0$, causing the weights to approach the zero vector: $\mathbf{w}^* \rightarrow \mathbf{0}$. This effect is called **shrinkage**.

(h) Limit as $\lambda \rightarrow 0$

As $\lambda \rightarrow 0$, the Ridge solution converges to $\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y}$.

- **Tall System (Overdetermined, $m > n$):** If X has full column rank, $X^T X$ is invertible. The solution becomes the standard **Ordinary Least Squares (OLS)** solution.
- **Wide System (Underdetermined, $m < n$):** The matrix $X^T X$ is singular and not invertible. The limit as $\lambda \rightarrow 0^+$ yields the **minimum-norm** solution to the least-squares problem.

Comment: All the answers to the subproblems are correct! Most impressively, Gemini handle the dimension of these complicated formulas accurately! I doubt that Gemini probably have seen these problems or topics from its corpus.

Interaction on Problem 5

Prompt: I'm a student learning deep learning. Please read this picture and extract the problem. Read the problem carefully, interpret them, and then tell me idea about them briefly.

Response:

Of course! It looks like you have a great set of problems covering foundational math and the mechanics of how a simple neural network learns. Here's a brief interpretation and the core ideas for each part.

Vector Calculus Review

These problems are a warm-up to make sure you're comfortable with the math needed for machine learning, specifically how to find gradients of common functions.

- **(a) - (e) Derivative Problems:**

- **Interpretation:** You're asked to find the derivative (gradient or Jacobian) of several linear and quadratic functions that appear constantly in ML. The key is to pay attention to the shapes of the inputs and outputs (scalar, vector, matrix).
- **Brief Idea:** 💡 The most straightforward way to solve these is to follow the instructions:
 1. Write out the function for an arbitrary component (e.g., the k -th element).
 2. Take the partial derivative with respect to that one component (e.g., $\partial/\partial x_k$).
 3. "Stack" these individual results together to form the final row vector or matrix. For example, for (b), the partial derivative of $\sum x_i^2$ with respect to x_k is just $2x_k$. Stacking these gives $[2x_1, 2x_2, \dots, 2x_n]$, which is $2\mathbf{x}^T$.

Least Squares and SVD

This section connects the classic problem of solving $X\mathbf{w} = \mathbf{y}$ to Singular Value Decomposition (SVD), a powerful tool that provides a stable solution whether you have more data than features or vice-versa.

- **(a-c) Overdetermined Case (Tall Matrix, $m > n$):**
 - **Interpretation:** This is the standard linear regression setup. You have more equations than unknowns, so there's no perfect solution. The goal is to find the **least-squares** solution—the one that minimizes the error.
 - **Brief Idea:** 🧠 The standard solution comes from the **normal equations**: $X^T X \mathbf{w} = X^T \mathbf{y}$. When you plug in the SVD ($X = U \Sigma V^T$) and use the fact that U and V are orthonormal ($U^T U = I, V^T V = I$), you'll find the solution is given by the **pseudoinverse** $X^\dagger$. You'll then show that $X^\dagger X = I$, meaning it acts as a **left-inverse**.
- **(d-f) Underdetermined Case (Fat Matrix, $m < n$):**
 - **Interpretation:** Here you have more features than data points, leading to infinite solutions. The goal is to find the "simplest" one, which is the solution with the **minimum norm** (shortest length).
 - **Brief Idea:** 🧠 This is a constrained optimization problem, typically solved with **Lagrange multipliers**. When you plug the SVD into this solution, you'll again find it's given by the pseudoinverse $X^\dagger$. This time, you'll show that $X X^\dagger = I$, meaning it acts as a **right-inverse**.

ReLU Elbow Update Under SGD

This section dives into the mechanics of how Stochastic Gradient Descent (SGD) actually works on a simple neural network with a ReLU activation function. The goal is to build intuition for how the network learns by changing the "shape" of the function it represents.

- **(a) Analyzing a Single ReLU:**
 - **Interpretation:** A warm-up to get familiar with the ReLU function, $\phi(z) = \max(0, z)$, and its derivatives.

- **Brief Idea:** ✅ The key is the chain rule. The "elbow" is where the input to the ReLU is zero. The derivative of the loss with respect to a parameter (like w or b) is calculated as: (derivative of loss w.r.t. ReLU output) $\times$ (derivative of ReLU output w.r.t. its input) $\times$ (derivative of ReLU input w.r.t. the parameter). Remember the derivative of ReLU is just 1 for positive inputs and 0 for negative inputs.
- **(b) Gradient Descent Dynamics & Drawing:**
 - **Interpretation:** This is the core intuitive part. Given a data point where the prediction is too high, how does an SGD step adjust the slope (w) and intercept (b) of the ReLU's linear part to "pull the function down"?
 - **Brief Idea:** 🍌 First, calculate the gradients for w and b at the given data point. The SGD update is `parameter = parameter - learning_rate * gradient`. Analyze the signs: if the prediction is too high, the error term ($\phi - y$) is positive. The sign of the full gradient will then depend on the signs of w and x . The update will push w and b in a direction that moves the "elbow" ($x = -b/w$) and changes the slope to reduce the error for that point. Your drawing should show the initial ReLU function, the data point, and the new, adjusted ReLU function after one SGD step.
- **(c) Full Network Elbow Location:**
 - **Interpretation:** Where are the "elbows" or non-linear points in the full, two-layer network function?
 - **Brief Idea:** The non-linearities in the network come from the ReLU activations in the first hidden layer. Each neuron in that layer has its own elbow. The elbows of the *entire network function* are located at the x values where the input to *any* of those ReLUs is zero.
- **(d) Deriving the New Elbow Location:**
 - **Interpretation:** Find a mathematical formula that describes where an elbow moves after one step of SGD on the full network.

- **Brief Idea:** ⚙️ This combines everything.

1. Start with the formula for an elbow's location: $e_j = -b_j / W_{j1}^{(1)}$.
2. Write down the update rules for b_j and $W_{j1}^{(1)}$ using SGD: $b_{j,\text{new}} = b_j - \lambda \frac{\partial \ell}{\partial b_j}$, etc.
3. You'll need to use the full **backpropagation** chain rule to find the gradients of the loss ℓ with respect to the first-layer parameters (b_j and $W_{j1}^{(1)}$).
4. Plug the updated parameters ($b_{j,\text{new}}$ and $W_{j1,\text{new}}^{(1)}$) back into the elbow formula to get the new location e'_j .

Comment: The AI seems to hallucinate with the last problem.....I will reopen a chat box, and ask it to answer the problem directly.

Prompt: I'm a student learning deep learning. Please read this picture and extract the problem. Read the problem carefully, and give me your accurate and brief answer.

Of course. Here are the solutions to the problems presented in the image.

(a) Derivatives of the Loss Function

The loss function is $\ell = \frac{1}{2}(\phi(x) - y)^2$, where $\phi(x) = \max(0, wx + b)$.

(i) Location of the 'elbow' e :

The elbow is where the function's behavior changes, which occurs when the input to the ReLU function is zero.

$$wx + b = 0 \implies x = -\frac{b}{w}.$$

So, the elbow is at $e = -\frac{b}{w}$.

(ii) Derivative of the loss w.r.t. $\phi(x)$:

Using the chain rule on $\ell = \frac{1}{2}(\phi(x) - y)^2$:

$$\frac{\partial \ell}{\partial \phi(x)} = \phi(x) - y$$

(iii) Partial derivative of the loss w.r.t. w :

Using the chain rule, $\frac{\partial \ell}{\partial w} = \frac{\partial \ell}{\partial \phi(x)} \frac{\partial \phi(x)}{\partial w}$. The derivative of ReLU, $\frac{\partial \phi(x)}{\partial w}$, is x if $wx + b > 0$ and 0 otherwise.

$$\frac{\partial \ell}{\partial w} = (\phi(x) - y) \cdot x \cdot \mathbb{I}(wx + b > 0)$$

where $\mathbb{I}(\cdot)$ is the indicator function.

(iv) Partial derivative of the loss w.r.t. b :

Similarly, $\frac{\partial \ell}{\partial b} = \frac{\partial \ell}{\partial \phi(x)} \frac{\partial \phi(x)}{\partial b}$. The derivative $\frac{\partial \phi(x)}{\partial b}$ is 1 if $wx + b > 0$ and 0 otherwise.

$$\frac{\partial \ell}{\partial b} = (\phi(x) - y) \cdot \mathbb{I}(wx + b > 0)$$

(b) Gradient Descent Behavior

Given a training point where the prediction $\phi(x)$ is 1 unit above the target y (i.e., $\phi(x) - y = 1$), the gradients simplify to $\frac{\partial \ell}{\partial w} = x \cdot \mathbb{I}(wx + b > 0)$ and $\frac{\partial \ell}{\partial b} = \mathbb{I}(wx + b > 0)$. The SGD update rule is $p' = p - \lambda \nabla_p \ell$.

(i) Case: $w > 0, x > 0, \phi(x) > 0$

Since $\phi(x) > 0$, the neuron is active.

- **Slope (w):** The gradient $\frac{\partial \ell}{\partial w} = x > 0$. The update is $w' = w - \lambda x$. The slope **decreases**.
- **Elbow (e):** The gradient $\frac{\partial \ell}{\partial b} = 1$. The update is $b' = b - \lambda$. The elbow's movement $e' = -\frac{b'}{w'}$ depends on the specific values. The new elbow location is $e' = -\frac{b-\lambda}{w-\lambda x}$. The elbow moves **right** if $w > bx$ and **left** if $w < bx$.

(ii) Case: $w > 0, x < 0, \phi(x) > 0$

The neuron is active.

- **Slope (w):** The gradient $\frac{\partial \ell}{\partial w} = x < 0$. The update is $w' = w - \lambda x$. The slope **increases**.
- **Elbow (e):** The gradient $\frac{\partial \ell}{\partial b} = 1$. The update is $b' = b - \lambda$. The elbow $e' = -\frac{b-\lambda}{w-\lambda x}$ will **move to the right**. This is because for the neuron to be active with $w > 0, x < 0$, we must have $x > e$, which implies $x > -b/w$, or $b > -wx > 0$. Since b, w, λ are positive and x is negative, the change in the elbow, $\Delta e = \frac{\lambda(w-bx)}{w(w-\lambda x)}$, will be positive.

(iii) Case: $w > 0, x > 0, \phi(x) = 0$

Since $\phi(x) = 0$, the neuron is inactive. The indicator function $\mathbb{I}(wx + b > 0)$ is 0.

- **Slope (w):** The gradient $\frac{\partial \ell}{\partial w} = 0$. There is **no change**.
- **Elbow (e):** The gradient $\frac{\partial \ell}{\partial b} = 0$. There is **no change**.
The parameters do not get updated for this input.

For the full network $\tilde{f}(x) = \mathbf{W}^{(2)}\Phi(\mathbf{W}^{(1)}x + \mathbf{b})$, the input to the i -th elementwise ReLU activation (the i -th neuron in the hidden layer) is $z_i = \mathbf{W}_{i,:}^{(1)}x + b_i$, where $\mathbf{W}_{i,:}^{(1)}$ is the i -th row of $\mathbf{W}^{(1)}$.

The elbow e_i is not a single point but a **hyperplane** in the input space $\mathbb{R}^d$ where the input to the neuron is zero. The equation for this hyperplane is:

$$\mathbf{W}_{i,:}^{(1)}x + b_i = 0$$

(d) New Elbow Location after SGD Update

After one SGD step with learning rate λ for a training pair (x_{train}, y) , the parameters for the i -th neuron, $\mathbf{W}_{i,:}^{(1)}$ and b_i , are updated. The new elbow location e'_i is the hyperplane defined by the updated parameters.

The updates are:

- $(\mathbf{W}_{i,:}^{(1)})' = \mathbf{W}_{i,:}^{(1)} - \lambda \cdot \delta_i \cdot x_{train}^T$
- $b'_i = b_i - \lambda \cdot \delta_i$

where the error signal $\delta_i = (\tilde{f}(x_{train}) - y)W_i^{(2)}\mathbb{I}(\mathbf{W}_{i,:}^{(1)}x_{train} + b_i > 0)$.

The new elbow location e'_i is defined by the hyperplane:

$$(\mathbf{W}_{i,:}^{(1)})'x + b'_i = 0$$

Substituting the updated values, the equation for the new hyperplane is:

$$\left(\mathbf{W}_{i,:}^{(1)} - \lambda\delta_i x_{train}^T\right)x + (b_i - \lambda\delta_i) = 0$$

Comment: The problem of a, b and c are answered correctly, but d problem is not correct. Gemini fail to analyse the problem based on multiple conditions and possibility. I think this is because subproblem d is never designed before, and Gemini has to do deduction and reasoning itself, and it failed to answer the question comprehensively.

Summary

Total score: 96 / 100

The accuracy of Gemini is amazing, especially for these classic problems that are highly likely to have appeared on the web forum or documents. For unseen questions, Gemini can provide inspiring insights, but still struggle to ace them all.

If input figures multiple times in one single chat, the AI may hallucinate across these figures. So for every single problem, it is suggestive to open a new box and communicate with LLM there.